

线性表 (Linear Lists)

本文件夹包含 5 种不同的线性表实现，每种都适用于不同的场景和需求。

📄 概览表

文件	实现方式	优点	缺点	适用场景
List.cpp	单向链表	灵活，支持排序	单向遍历	通用场景，需要排序
CircList.cpp	循环链表	环形结构，易于循环	较复杂	循环处理，如约瑟夫问题
DblList.cpp	双向链表	双向遍历	多占内存	需要双向操作的场景
SeqList.cpp	顺序表	随机访问，缓存友好	插删效率低	频繁查询，插删少
StaticList.cpp	静态链表	无指针，内存连续	空间固定	嵌入式系统，栈内分配

📖 详细说明

1 List.cpp - 单向链表 (推荐)

特点:

- 带虚拟头结点的链表设计
- 支持完整的增删改查操作
- 集成归并排序算法
- 模板设计，支持任意数据类型

主要方法:

```
int Length()           // 获取链表长度
bool Insert(size_t index, T value) // 在指定位置插入
bool Remove(size_t index, T &value) // 删除指定位置元素
LinkNode<T>* Search(const T &value) // 查找元素
void Sort()            // 归并排序
T& operator[](size_t index) // 下标访问
```

构造函数:

```
List()                // 创建空链表
List(const T &value)   // 创建包含一个元素的链表
List(const initializer_list<T> &) // 用初始化列表创建
List(const List<T> &other) // 复制构造
```

时间复杂度:

- 查询: $O(n)$
- 插入/删除: $O(n)$
- 排序: $O(n \log n)$

使用示例:

```
List<int> list;
list.input(50, 30, 10, 40, 20);
list.output();           // 50 30 10 40 20
list.Sort();             // 归并排序
list.output();           // 10 20 30 40 50
list.Insert(2, 25);      // 在索引2处插入25
int val;
list.getData(1, val);    // 获取索引1的值
list.Remove(1, val);     // 删除索引1, 值保存到val
```

2] CircList.cpp - 循环链表

特点:

- 尾结点的 `next` 指向头结点 (包括虚拟头)
- 适合需要循环处理的场景
- 虚拟头结点的 `data` 域存储链表大小

主要操作:

```
bool Insert(size_t i, const T &e)
bool Delete(size_t i, T &e)
bool GetElem(size_t i, T &e) const
int Length() const
void output() const
```

特殊用途:

```
// 约瑟夫问题
// n个人围成一圈, 每数k个人就删除一个
// 循环链表最适合此场景
```

时间复杂度:

- 查询: $O(n)$
- 插入/删除: $O(n)$

使用示例:

```
CircList<int> clist;
clist.Insert(0, 1);
clist.Insert(1, 2);
clist.Insert(2, 3);
clist.output();           // 1 2 3
int val;
clist.GetElem(0, val);    // val = 1
```

3 DbList.cpp - 双向链表

特点:

- 每个结点有 **prior** (前驱) 和 **next** (后继) 指针
- 支持双向遍历
- 插删操作需要更新两个指针

结点结构:

```
struct LinkNode {
    T data;
    LinkNode *prior;
    LinkNode *next;
};
```

主要方法:

```
bool Insert(size_t i, const T &e)    // 在位置i插入
bool Delete(size_t i, T &e)          // 删除位置i
bool GetElem(size_t i, T &e) const    // 获取位置i的值
LinkNode<T>* Search(const T &e) const // 查找元素
```

时间复杂度:

- 查询: $O(n)$
- 插入/删除: $O(n)$
- 空间: $O(n)$ (多占约 8 字节/结点)

使用示例:

```
DbList<int> dlist;
dlist.Insert(0, 10);
dlist.Insert(1, 20);
dlist.Insert(2, 30);
dlist.output();           // 10 20 30
```

```
int val;
dlist.GetElem(1, val);    // val = 20
dlist.Delete(1, val);     // 删除20
```

4 SeqList.cpp - 顺序表

特点:

- 基于动态数组实现
- **随机访问** $O(1)$ ⚡
- 插删效率低 $O(n)$
- 缓存友好

数据结构:

```
template <typename T>
class SeqList {
private:
    T *data;        // 动态数组
    int Length;     // 当前长度
    int MaxSize;    // 最大容量
};
```

主要方法:

```
bool Insert(size_t i, const T &e)    //  $O(n)$ 
bool Delete(size_t i, T &e)          //  $O(n)$ 
bool GetElem(size_t i, T &e)         //  $O(1)$  ★
T& operator[](size_t i)              //  $O(1)$  ★
int GetLength() const                //  $O(1)$ 
```

时间复杂度:

操作	复杂度
随机访问	$O(1)$
顺序查找	$O(n)$
插入	$O(n)$
删除	$O(n)$

使用示例:

```
SeqList<int> slist(10);    // 容量为10
slist.Insert(0, 5);
```

```
slist.Insert(1, 10);
slist.Insert(2, 15);

int val = slist[1];           // 0(1) 随机访问
slist.GetElem(0, val);        // val = 5

// 频繁插删会很慢，不推荐
```

何时使用：

- ☒ 频繁随机访问
- ☒ 很少插删
- ☒ 数据量确定
- ☒ 频繁插删则不推荐

5 StaticList.cpp - 静态链表

特点：

- 用**数组模拟链表**，无指针
- 所有结点在同一数组中
- 内存连续，缓存友好
- 不需要动态内存分配

结点结构：

```
struct Node {
    T data;
    int next;           // 下一个结点的数组索引，-1表示结束
};
```

原理：

空闲链表：-1 -> 1 -> 2 -> 3
实际链表：0 -> 2 -> 4 -> -1

数组：

0: [data, next=2]	← 链表首结点
1: [data, next=3]	← 空闲
2: [data, next=4]	← 链表第二结点
3: [data, next=-1]	← 空闲
4: [data, next=-1]	← 链表末结点

主要方法：

```
bool Insert(size_t i, const T &e)
bool Delete(size_t i, T &e)
int Locate(const T &e)           // 定位元素
int Length()
void output()
```

特点:

- 无需 new/delete
- 内存预分配，快速
- 适合嵌入式系统

使用示例:

```
StaticList<int> stlist(100); // 100个元素的空间
stlist.Insert(0, 10);
stlist.Insert(1, 20);
stlist.Insert(2, 30);

int val;
stlist.Locate(20);           // 查找元素20
stlist.output();             // 10 20 30
```

何时使用:

- ☒ 嵌入式系统
- ☒ 不需要动态分配
- ☒ 内存受限
- ☒ 空间预知

对比总结

特性	List	CircList	Dbllist	SeqList	StaticList
随机访问	O(n)	O(n)	O(n)	O(1)	O(n)
插入	O(n)	O(n)	O(n)	O(n)	O(n)
删除	O(n)	O(n)	O(n)	O(n)	O(n)
排序	☒	✗	✗	☒	✗
双向遍历	✗	✗	☒	☒	✗
内存动态	☒	☒	☒	☒	✗
推荐度	★★★★★	★★★★	★★★★★	★★★★★	★★★

编译与运行

编译单个文件

```
# 编译单向链表
g++ -Wall -Wextra -g3 List.cpp -o output/List.exe

# 编译循环链表
g++ -Wall -Wextra -g3 Circlist.cpp -o output/Circlist.exe

# 编译双向链表
g++ -Wall -Wextra -g3 Dbllist.cpp -o output/Dbllist.exe

# 编译顺序表
g++ -Wall -Wextra -g3 SeqList.cpp -o output/SeqList.exe

# 编译静态链表
g++ -Wall -Wextra -g3 StaticList.cpp -o output/StaticList.exe
```

运行

```
cd output
List.exe
Circlist.exe
Dbllist.exe
SeqList.exe
StaticList.exe
```

学习建议

初学者

1. 先学习 **List.cpp** (单向链表基础)
2. 然后学习 **SeqList.cpp** (对比数组)
3. 理解指针和链表的关系

中级

4. 学习 **Dbllist.cpp** (双指针操作)
5. 学习 **Circlist.cpp** (环形结构)
6. 理解各种链表的特殊应用

高级

7. 学习 **StaticList.cpp** (指针模拟)
 8. 理解空间效率和时间效率的权衡
 9. 在实际项目中选择合适的数据结构
-

🔗 实战场景

场景1：需要快速查询和排序

```
// ☒ 使用 List (单向链表)
List<int> list;
list.input(50, 30, 10, 40, 20);
list.Sort();    // 归并排序
```

场景2：需要频繁随机访问

```
// ☒ 使用 SeqList (顺序表)
SeqList<int> slist(1000);
for(int i = 0; i < 1000; i++) {
    slist[i] = i;    // O(1)
}
```

场景3：需要双向遍历

```
// ☒ 使用 Dbllist (双向链表)
Dbllist<int> dlist;
dlist.Insert(0, 1);
dlist.Insert(1, 2);
// 可以向前向后遍历
```

场景4：实现约瑟夫问题

```
// ☒ 使用 CircList (循环链表)
CircList<int> circle;
// 完美的环形结构
```

场景5：嵌入式系统

```
// ☒ 使用 StaticList (静态链表)
StaticList<int> stlist(256);    // 固定大小，栈分配
```

🔍 常见问题

Q: 应该选择哪个? A: 99% 的情况下选择 **List.cpp** (单向链表)。除非有特殊需求。

Q: List 和 SeqList 如何选择? A:

- 频繁查询 → **SeqList**
- 频繁插删 → **List**
- 都要做 → **DbList** (折中)

Q: StaticList 什么时候用? A: 嵌入式系统或不允许动态内存分配的场景。

Q: 为什么 List 支持排序? A: 因为链表虽然查询慢, 但链表适合**归并排序** (不需要随机访问)。

参考资料

- **关键概念**: 指针、动态内存、链表、堆、队列
- **算法**: 遍历、搜索、排序、插删
- **复杂度**: 时间复杂度、空间复杂度、Big-O 记法

最后更新: 2026年1月18日 作者: 数据结构学习项目